

Department of Electrical and Computer Engineering
University of Wisconsin – Madison
ECE 552 Introduction to Computer Architecture
Homework 1 (Due 9/15)

1. *Translating C statement to MIPS assembly code*

In the following sub-problems, assume variables f, g, h, i, and j are stored in registers \$s0, \$s1, \$s2, \$s3, and \$s4 respectively. Assume that the base address of int arrays A and B are in registers \$s6, and \$s7 respectively. For the C statement in each sub-problem, translate it into corresponding MIPS code using the minimum number of assembly instructions.

(a) $f = g + (-5 - f)$

Answer:

sub \$s0, \$zero, \$s0 addi \$s0, \$s0, -5 add \$s0, \$s0, \$s1	addi \$s0, \$s0, 5 sub \$s0, \$s1, \$s0
--	--

(b) $B[8] = A[j-i]$ // assume $j \geq i$

Answer:

```

sub $t0, $s4, $s3    # j-i
sll $t0, $t0, 2      # 4*(j-i)
add $t0, $s6, $t0    # A[0] base address + 4*(j-i)
lw $t1, 0($t0)       # $t1 <- A[j-i]
sw $t1, 32($s7)      # B[8] <- $t1

```

2. *Translating MIPS assembly code to C statements*

In the following sub-problems, assume variables f, g, h, i, and j are stored in registers \$s0, \$s1, \$s2, \$s3, and \$s4 respectively and their values are multiples of 4. Assume that &A (the base address of int array A) and &B (the base address of int array B) are in registers \$s6, and \$s7 respectively. For the MIPS assembly code in each sub-problem, write down the corresponding C statements.

(a)

```

add $t0, $s6, $s0
add $t1, $s7, $s1
lw $s0, 0($t0)
addi $t2, $t0, 4
lw $t0, 0($t2)
add $t0, $t0, $s0
sw $t0, 0($t1)

```

Answer:

```

add $t0, $s6, $s0    # $t0 <- &A + f
add $t1, $s7, $s1    # $t1 <- &B + g
lw $s0, 0($t0)       # $s0 <- A[f/4]

```

```

addi $t2, $t0, 4      # $t2 <- &A + f + 4
lw   $t0, 0($t2)      # $t0 <- A[f/4 + 1]
add  $t0, $t0, $s0     # $t0 <- A[f/4 + 1] + A[f/4]
sw   $t0, 0($t1)      # B[g/4] <- A[f/4 + 1] + A[f/4]

```

Thus,
 $B[g/4] = A[f/4] + A[1+f/4];$
 $f = A[f/4];$

(b)

```

addi $t0, $s6, 4
add  $t1, $s6, $zero
sw   $t1, 0($t0)
lw   $t0, 0($t0)
add  $s0, $t1, $t0

```

Answer:

```

addi $t0, $s6, 4      # $t0 <- &A + 4
add  $t1, $s6, $zero # $t1 <- &A
sw   $t1, 0($t0)      # A[1] <- &A
lw   $t0, 0($t0)      # $t0 <- A[1] = &A
add  $s0, $t1, $t0    # f = 2*&A

```

Thus,
 $A[1] = \&A[0];$
 $f = 2*(\&A[0]);$

3. *Simulating MIPS assembly programs*

- (a) Assume that the contents of registers \$t0 and \$t1 are $[t0] = 0xF00DD00D$, and $[t1] = 0x21212121$ respectively. What is the value \$t2 after execution of the following sequence of MIPS instructions?

```

sll  $t2, $t0, 4
addi $t2, $t2, -1

```

Answer:

After logic shift left by 4, $[t2] = 0x00DD00D0$. Then, $[t2]$ is subtracted by 1, so $[t2] = 0x00DD00D0 - 0x00000001 = 0x00DD00CF$

- (b) Suppose that $[t0] = 0x7F200EE0$, and $[t1] = 0x11F80020$. What is the value of \$t2 after execution of the following instructions?

```

        slt $t2, $zero, $t0
        bne $t2, $zero, ELSE
        j   DONE
ELSE:    addi $t2, $t2, 2
DONE:

```

Answer:

After slt, $[t2] = 1$ since $0 < [t0]$. Thus, branch to ELSE will take place. This will add 2 to $[t2]$, making $[t2] = 3$ after execution.

4. *Pseudo-instructions*

Consider a hypothetical MIPS pseudo-instruction

orn rd, rs, rt

that will perform the bit-wise OR of rt and $\sim$ rs (the bit-wise complement of rs) and put the result into rd. Use the minimum number of MIPS instructions to synthesize this pseudo-instruction.

Answer:

```
nor $t1, rs, rs # or nor $t1, rs, $zero  
or rd, $t1, rt
```

5. *Branch instructions*

(a) Consider the following sequence of MIPS instructions

```
    add $t1, $zero, $zero  
LOOP: lw  $s1, 0($s0)  
      add $s2, $s2, $s1  
      addi $s0, $s0, 4  
      addi $t1, $t1, 1  
      slti $t2, $t1, 50  
      bne $t2, $zero, LOOP
```

Translate these instructions to a C program segment assuming integer variable X is in register \$t1, and \$s2 holds the integer variable Result, and \$s0 holds the base address of the integer array MemArray.

Answer:

```
X = 0;  
do {  
    Result += MemArray[X];  
    X++;  
} while (X < 50);
```

(b) Rewrite the loop to reduce the number of MIPS instructions executed.

Answer:

```
    addi $t1, $s0, 200 # pre-compute 1-word-beyond-the-end  
LOOP: lw $s1, 0($s0)  
      add $s2, $s2, $s1  
      addi $s0, $s0, 4  
      bne $t1, $s0, LOOP # repeat only if we're within
```